
Données, fine-tuning et diagnostics du système hybride

Support technique pour la présentation PowerPoint

Système hybride de recommandation emploi-compétences

Ce document rassemble les éléments à expliquer au jury : transformation des données, niveaux NCF, preuve du fine-tuning, paramètres optimisés, distinction avec LoRA, calibration bayésienne, structure du graphe, dépendance aux hubs et limites de validité.

Table des matières

1	Préparation et normalisation des données	2
1.1	Chaîne de traitement	2
1.2	Exemple simple	2
2	Niveaux de la NCF utilisés par le système	3
3	Le modèle a-t-il réellement été fine-tuné ?	3
4	Protocole d'entraînement	5
5	Quels paramètres ont été optimisés ?	5
6	Pourquoi ce n'est pas du LoRA	7
7	Résultats avant et après fine-tuning	7
8	Optimisation bayésienne avec l'échantillonneur TPE	9
8.1	Principe général	9
8.2	Application au score hybride	9
9	Structure du graphe et dépendance aux hubs	11
9.1	Comptages Cypher et projection matching	11
9.2	Degré et identification des hubs	11
9.3	Densité et composante géante	12
9.4	Test de robustesse par retrait ciblé	12
9.5	Protection contre le bruit de popularité	12
10	Trame recommandée pour le PowerPoint	13
11	Questions probables du jury	13
12	Sources techniques vérifiées	14

Préparation et normalisation des données

La normalisation transforme des offres et des profils hétérogènes en informations comparables. Elle ne crée pas une information absente : elle nettoie, organise et code ce qui est disponible.

Chaîne de traitement

1. **Nettoyer les textes** : suppression des espaces inutiles, valeurs vides explicites et résidus provenant du scraping.
2. **Uniformiser les écritures** : par exemple, « douala », « DOUALA » et « Douala » sont ramenés à une modalité commune.
3. **Séparer les valeurs multiples** : « Python ; Excel / SQL » devient une liste de trois compétences.
4. **Coder les qualifications** : les diplômes et niveaux d'études sont rapprochés des niveaux de la Nomenclature camerounaise des formations (NCF).
5. **Construire le texte d'encodage** : le champ `text_to_embed` réunit le titre, les compétences, le secteur, le niveau, l'expérience et la description disponibles.

Exemple simple

Donnée brute	« Data analyst ; douala ; Bac+5 ; Python/Excel/SQL »
Donnée normalisée	Métier : Data Analyst ; ville : Douala ; niveau NCF : 8 ; compétences : Python, Excel et SQL.

Les données normalisées alimentent deux représentations complémentaires : la mémoire vectorielle compare le sens global des textes ; le graphe crée des relations explicites telles que CANDIDAT--POSSEDE--COMPETENCE et OFFRE--REQUIERT--COMPETENCE.

Point de vigilance

Le pipeline ne supprime pas automatiquement toutes les lignes répétées. Il audite les doublons et conserve les observations ; il retire surtout les répétitions internes aux listes, par exemple [Python, Python, Excel] devient [Python, Excel].

Formulation orale conseillée

« La normalisation met les offres et les candidats dans un langage commun. Elle rend les champs comparables avant l'encodage vectoriel et avant la création des relations du graphe. »

Niveaux de la NCF utilisés par le système

Les codes NCF ne sont pas des notes. Ils ordonnent les types de formation et facilitent la comparaison entre le niveau d'un candidat et celui demandé par une offre.

Code	Niveau de formation	Exemples
1	Formation sur le tas	Apprentissage pratique, attestation
2	Formation courte en centre agréé	Formation généralement inférieure à un an
3	Primaire et post-primaire	CEP, SAR/SM
4	Secondaire premier cycle	BEPC, CAP, BEP
5	Secondaire second cycle	Baccalauréat, agent technique
6	Postsecondaire court, Bac+2	BTS, DUT, HND, DEUG
7	Supérieur premier cycle, Bac+3	Licence, ingénieur des travaux
8	Supérieur second cycle, Bac+4 ou Bac+5	Master I et II, maîtrise, ingénieur de conception
9	Niveau doctoral, Bac+6 et plus	Doctorat, PhD

Ainsi, un Master ou Bac+5 correspond au **niveau NCF 8**, une Licence au niveau 7, un BTS/DUT au niveau 6 et un Doctorat au niveau 9. Pour les candidats, le diplôme déclaré, plus précis, est utilisé en priorité ; le niveau d'étude général sert de solution de repli.

Point de vigilance

Le fichier de configuration classe actuellement **Master 1** au niveau 7, alors que la NCF chargée dans le projet place Master I et II au niveau 8. Ce mapping doit être corrigé avant une nouvelle exécution complète, sinon certaines compatibilités de qualification seront sous-estimées.

Le modèle a-t-il réellement été fine-tuné ?

Oui. Un fine-tuning suppose que l'on parte d'un modèle préentraîné, que l'on calcule une perte sur des données propres au domaine, puis que l'on modifie ses poids par rétropropagation. Les artefacts du projet montrent ces trois éléments.

- modèle de départ : `sentence-transformers/all-MiniLM-L6-v2` ;
- appel effectif à `trainer.train()` ;
- checkpoints contenant les poids, l'optimiseur et le scheduler ;

- cinq époques et 710 étapes enregistrées ;
- modèle final sauvegardé dans `models/st_finetuned/final` ;
- perte observée passant d'environ 2,45 au début à environ 0,13 en fin d'entraînement.

Formulation orale conseillée

« Il s'agit d'un vrai fine-tuning, car les poids du Transformer ont été optimisés sur des paires du domaine emploi. La présence des checkpoints, des gradients, de l'état de l'optimiseur et du modèle final exclut une simple réévaluation du modèle de base. »

Protocole d'entraînement

Élément	Valeur retenue
Modèle initial	all-MiniLM-L6-v2
Architecture	6 couches Transformer, 12 têtes d'attention
Dimension des embeddings	384
Paires d'entraînement	4 542
Paires de validation	981
Paires de test	953
Époques	5
Taille de lot	32
Taux d'apprentissage	2×10^{-5}
Perte	Multiple Negatives Ranking Loss
Scheduler	Cosine avec warmup
Weight decay	0,01
Critère du meilleur checkpoint	NDCG@10 de validation
Meilleur checkpoint	Époque 4, NDCG@10 = 0,6492
Durée d'entraînement enregistrée	15 157 secondes, soit 4 h 12 min 37 s, sur CPU

La perte contrastive rapproche la représentation des métadonnées d'une offre de celle de sa description. Dans un lot de 32 paires positives, les autres descriptions du lot servent de négatifs implicites. Le modèle apprend donc à distinguer la bonne description des descriptions concurrentes sans nécessiter une annotation négative manuelle pour chaque paire.

Formulation orale conseillée

« Le fine-tuning complet sur cinq époques a duré 15 157 secondes, soit environ 4 heures et 13 minutes sur CPU. Cette durée correspond à l'entraînement enregistré ; elle n'inclut pas toute la préparation des données ni les analyses réalisées après l'entraînement. »

Quels paramètres ont été optimisés ?

Le modèle final contient **22 713 216 paramètres**. L'audit du modèle montre que **100 % de ces paramètres étaient entraîna**bles. Ils appartiennent à l'encodeur :

- embeddings des tokens et des positions ;
- matrices Query, Key et Value de l'attention ;
- projections de sortie de l'attention ;

- couches feed-forward ;
- paramètres des LayerNorm des six couches Transformer.

Le mean pooling calcule la moyenne des représentations des tokens et la dernière étape normalise le vecteur. Ces deux modules ne contiennent aucun poids entraînable. Le tokenizer et son vocabulaire de 30 522 tokens n'ont pas été réentraînés.

Point de vigilance

Les paramètres appris ne doivent pas être confondus avec les hyperparamètres. Les poids du Transformer sont appris automatiquement ; le nombre d'époques, la taille de lot et le taux d'apprentissage sont fixés avant l'entraînement.

Pourquoi ce n'est pas du LoRA

Aspect	Fine-tuning réalisé	LoRA
Poids initiaux	Tous entraînaables	Généralement gelés
Paramètres ajoutés	Aucun adaptateur	Petites matrices de faible rang
Part entraînée	100 % des 22,7 millions	Faible proportion des paramètres
Bibliothèque PEFT	Non utilisée	Habituellement utilisée
Sortie	Modèle complet sauvegardé	Adaptateurs séparés ou fusionnés

LoRA approxime la mise à jour d'une matrice W par deux petites matrices :

$$W' = W + BA,$$

où le rang de BA est faible. Cette stratégie réduit la mémoire nécessaire pour adapter les grands modèles. Elle n'a pas été utilisée ici : aucune couche n'est gelée et aucun adaptateur de faible rang n'est injecté.

Formulation orale conseillée

« Nous avons effectué un fine-tuning complet, et non du LoRA. Ce choix reste réalisable parce que MiniLM est un encodeur compact de 22,7 millions de paramètres, très inférieur à la taille des grands modèles génératifs. »

Résultats avant et après fine-tuning

Métrique	Modèle initial	Modèle fine-tuné	Gain absolu
NDCG@10	0,1681	0,6232	+0,4552
MRR@10	0,1410	0,5874	+0,4463
Recall@1	0,0986	0,5268	+0,4281
Recall@5	0,1983	0,6632	+0,4648
Recall@10	0,2560	0,7398	+0,4837

Le Recall@10 de 0,7398 signifie que le modèle retrouve environ 74 % des descriptions attendues parmi les dix premiers résultats du protocole de test. Le NDCG@10 de 0,6232 indique que les bons résultats sont aussi mieux placés en tête de liste qu'avec le modèle initial.

Point de vigilance

Cette évaluation mesure une tâche de récupération construite à partir des offres : retrouver la description correspondant aux métadonnées de la même offre. Elle démontre une adaptation sémantique au domaine, mais pas encore une pertinence candidat-offre validée par des recruteurs. Les chiffres ne doivent donc pas être présentés comme une preuve directe de qualité du recrutement.

Optimisation bayésienne avec l'échantillonneur TPE

Le terme correct est **échantillonneur TPE**, et non « échantillon TPE ». TPE signifie *Tree-structured Parzen Estimator*. Dans Optuna, cet algorithme choisit les paramètres à tester au prochain essai en exploitant les résultats déjà observés.

Principe général

Après quelques essais initiaux, TPE sépare les combinaisons évaluées en deux groupes :

- les essais ayant obtenu les meilleurs résultats ;
- les essais ayant produit des résultats moins bons.

Il estime ensuite deux distributions :

$$l(w) = p(w \mid \text{bon résultat}) \quad \text{et} \quad g(w) = p(w \mid \text{résultat moins bon}).$$

TPE privilégie les paramètres pour lesquels le rapport suivant est élevé :

$$\frac{l(w)}{g(w)}.$$

Il recherche donc des valeurs fréquentes parmi les bons essais et rares parmi les mauvais. L'algorithme ne calcule pas directement la solution optimale : il apprend progressivement dans quelle zone effectuer les prochains essais, tout en conservant une part d'exploration.

Application au score hybride

Pour chaque couple candidat-offre, le score calibré prend la forme :

$$S(c, o) = \alpha S_{\text{sem}}(c, o) + \beta S_{\text{Neo4j}}(c, o), \quad \alpha + \beta = 1.$$

À chaque essai, Optuna propose deux poids bruts, les normalise, recalcule le score des offres, classe les résultats de chaque candidat et mesure le NDCG@10 moyen. Le processus réalisé dans le projet comprend 500 essais sur 690 couples candidat-offre associés à 69 candidats, avec une graine fixée à 42.

Poids Neo4j	NDCG@10	Lecture simplifiée
0,10	0,82	Faible performance observée
0,30	0,87	Amélioration du classement
0,70	0,94	Zone prometteuse
0,80	0,96	Bon essai ; TPE explorera probablement autour de cette zone

Cet exemple est pédagogique : ces quatre valeurs ne constituent pas les résultats bruts du projet. Elles illustrent seulement la manière dont TPE déplace progressivement sa recherche.

Point de vigilance

L'optimisation utilise ici **proxy_relevance** comme vérité de référence. Elle recherche donc les poids qui reproduisent au mieux ce proxy, et non une pertinence certifiée par des recruteurs. Le résultat doit être confirmé sur un jeu indépendant annoté par des experts. De plus, la solution exacte « Neo4j seul » a été évaluée comme cas limite après les essais TPE, car l'espace de recherche imposait un poids strictement positif à chaque composante.

Formulation orale conseillée

« L'échantillonneur TPE est une méthode d'optimisation bayésienne qui modélise séparément les bonnes et les mauvaises combinaisons de poids. Il propose ensuite les essais ayant la plus forte probabilité d'améliorer le NDCG@10, sans devoir tester uniformément toutes les combinaisons. »

Structure du graphe et dépendance aux hubs

Cette analyse vérifie si le graphe peut relier candidats, compétences et offres sans dépendre excessivement de quelques nœuds génériques. Elle intervient avant le reranking : un graphe trop fragmenté explique mal les recommandations ; un graphe dominé par des hubs produit au contraire des rapprochements trop larges.

Comptages Cypher et projection matching

Des requêtes Cypher comptent directement dans Neo4j les nœuds par label, les relations par type et le nombre de voisins de chaque nœud. Le graphe complet contient 90 230 nœuds et 550 542 relations.

Une **projection matching** est ensuite construite en conservant les relations utiles : POSSEDE, REQUIERT, NECESSITE, CORRESPOND_MEPC, A_NIVEAU, REQUIERT_NIVEAU_NCF, DANS_SECTEUR et LOCALISEE_A. Cette projection compte 72 516 nœuds et 499 080 arêtes.

Pour étudier la connectivité, elle est transformée en graphe simple non dirigé avec NetworkX. Le diagnostic vérifie alors l'existence d'une connexion sans interpréter son orientation.

Point de vigilance

La projection non dirigée convient au diagnostic de structure, mais pas à l'interprétation métier finale : elle ignore le sens des relations et peut fusionner plusieurs liens entre les mêmes nœuds.

Degré et identification des hubs

Le degré d'un nœud est son nombre de voisins. Les statistiques calculées par type de nœud sont la moyenne, la médiane, le minimum, le maximum et la variance. Pour les compétences, le degré médian est de 4, alors que le maximum atteint 6 787. Les niveaux NCF atteignent jusqu'à 10 261 connexions.

La distribution des degrés est représentée sur des axes logarithmiques afin de rendre visibles simultanément les nombreux nœuds faiblement connectés et les quelques hubs. La longue traîne observée signifie que la connectivité est très concentrée.

Indicateur	Interprétation
Degré élevé	Le nœud est relié à de nombreuses entités ; il facilite la navigation.
Hub générique	Le nœud relie beaucoup d'objets mais discrimine peu les offres.
Degré médian faible	La majorité des nœuds possède seulement quelques relations ciblées.
Variance élevée	Les connexions sont très inégalement réparties entre les nœuds.

Un degré élevé n'est donc pas une preuve de pertinence. Une compétence générale comme la communication peut être fréquente sans suffire à justifier une recommandation précise.

Densité et composante géante

Pour un graphe non dirigé comportant n nœuds et m arêtes, la densité est :

$$D = \frac{2m}{n(n-1)}.$$

Une faible densité n'est pas automatiquement un défaut : une offre doit être reliée uniquement aux compétences, niveaux et secteurs qui la décrivent, et non à toutes les entités du graphe.

La composante géante est le plus grand groupe de nœuds accessibles les uns depuis les autres. Dans la projection initiale, elle contient 72 516 nœuds. Les principaux chemins candidat-compétence-offre sont donc disponibles pour le GraphRAG.

Test de robustesse par retrait ciblé

Les nœuds sont classés par degré décroissant. Le programme retire ensuite les 1 %, 5 % et 10 % les plus connectés, puis recalcule la taille de la composante géante.

Hubs retirés	Composante géante restante	Lecture
0 %	100 %	Projection initiale connectée
1 %	30,1 %	Forte dépendance aux nœuds les plus connectés
5 %	11,3 %	Fragmentation importante
10 %	0,07 %	Effondrement presque total de la projection

Il s'agit d'une attaque ciblée et non d'une panne aléatoire. Le résultat montre une dépendance structurelle aux hubs, mais ne prouve pas à lui seul que Neo4j est inutilisable. Le retrait touche aussi des nœuds organisateurs comme les niveaux NCF, les secteurs et les localisations ; une forte fragmentation est donc partiellement attendue.

Protection contre le bruit de popularité

Le score Neo4j n'utilise pas le degré brut. Il mesure la couverture des compétences :

$$S_{\text{Neo4j}}(c, o) = \frac{|\mathcal{C}_c \cap \mathcal{C}_o|}{|\mathcal{C}_o|},$$

où $\mathcal{C}_c$ désigne les compétences du candidat et $\mathcal{C}_o$ celles requises par l'offre. Une compétence fréquente ne compte que si elle est possédée par le candidat et demandée par l'offre. Cette règle limite le biais de popularité dans le reranking.

Formulation orale conseillée

« L'analyse combine des comptages Cypher, une projection NetworkX, la distribution des degrés, la composante géante et un retrait ciblé des hubs. Le graphe est connecté mais dépend de quelques nœuds génériques. Nous utilisons donc la couverture des compétences, et non le degré brut, comme signal Neo4j du score hybride. »

Trame recommandée pour le PowerPoint

1. **Diapositive 1 – Pourquoi normaliser ?** Montrer un exemple brut puis normalisé.
2. **Diapositive 2 – Niveaux NCF.** Présenter uniquement les niveaux 5 à 9 les plus mobilisés dans les offres, avec Bac+5 → niveau 8.
3. **Diapositive 3 – Données de fine-tuning.** Afficher les trois tailles de split et la construction métadonnées → description.
4. **Diapositive 4 – Entraînement.** Présenter modèle initial, perte contrastive, cinq époques, batch 32 et meilleur checkpoint.
5. **Diapositive 5 – Paramètres entraînés.** Insister sur 22,7 millions de paramètres, 100 % entraînaibles, et préciser « pas de LoRA ».
6. **Diapositive 6 – Résultats.** Comparer NDCG@10, MRR@10 et Recall@10 avant/après.
7. **Diapositive 7 – Calibration TPE.** Montrer la boucle proposition des poids, classement, calcul du NDCG@10 et nouvel essai.
8. **Diapositive 8 – Structure du graphe.** Présenter degré, hubs et composante géante.
9. **Diapositive 9 – Robustesse.** Montrer l'effet du retrait ciblé et expliquer pourquoi le score n'utilise pas le degré brut.
10. **Diapositive 10 – Limites.** Expliquer que le test valide le retrieval d'offres et que l'optimisation dépend d'un proxy ; une annotation humaine reste nécessaire.

Questions probables du jury

Pourquoi avoir fine-tuné MiniLM ?

Le vocabulaire et les formulations des offres camerounaises diffèrent des textes généraux sur lesquels le modèle initial a été préentraîné. L'adaptation contrastive améliore leur représentation.

Pourquoi cinq époques ?

Les métriques de validation progressent jusqu'à l'époque 4, puis reculent légèrement à l'époque 5. Le mécanisme de meilleur checkpoint conserve donc l'état de l'époque 4. Les cinq époques servent à observer cette stabilisation, pas à affirmer que la dernière époque est nécessairement

la meilleure.

Pourquoi ne pas utiliser LoRA ?

Le modèle ne contient que 22,7 millions de paramètres. Un fine-tuning complet reste possible sur la machine disponible et permet d'adapter directement tout l'encodeur.

Le modèle apprend-il directement le matching candidat-offre ?

Non. Il apprend d'abord une représentation adaptée aux offres. Le matching final combine ensuite la similarité vectorielle et les relations du graphe. Une validation humaine indépendante reste nécessaire pour mesurer la pertinence métier réelle.

Comment TPE choisit-il le prochain essai ?

Il compare les distributions des paramètres associés aux bons et aux moins bons résultats, puis privilégie les valeurs fréquentes dans le premier groupe et rares dans le second. Il maintient toutefois une part d'exploration afin de ne pas rester bloqué trop tôt dans une zone sous-optimale.

Un hub très connecté est-il une compétence très pertinente ?

Non. Le degré mesure la connectivité, pas la pertinence pour un candidat donné. Une compétence générale peut être reliée à de nombreuses offres tout en étant peu discriminante. Le score utilise donc la proportion des compétences requises effectivement possédées.

Sources techniques vérifiées

- Configuration : `src/02_finetune_st/config_st.json`.
- Entraînement : `src/02_finetune_st/train_sentence_transformer.py`.
- Données : `data/finetune/pairs_train.jsonl`, `pairs_val.jsonl` et `pairs_test.jsonl`.
- État du trainer : `models/st_finetuned/checkpoints/checkpoint-710/trainer_state.json`.
- Modèle final : `models/st_finetuned/final/model.safetensors`.
- Comparaison : `models/st_finetuned/eval_comparatif/evaluation_comparatif.json`.
- Calibration TPE : `src/07_evaluation/optimize_hybrid_weights.py`.
- Résultats Optuna : `outputs/evaluation/hybrid_weight_optimization/best_weights_ndcg.json`.
- Diagnostic du graphe : `scripts/generate_neo4j_network_diagnostics.py`.
- Robustesse : dossier `outputs/memoire_stats/implementation_deep_current`, fichier `18_neo4j_robustesse.csv`.
- Référentiel NCF : `data/processed/ncf_niveaux.parquet`.
- Référence scientifique : Reimers, N. et Gurevych, I. (2019), *Sentence-BERT : Sentence*

Embeddings using Siamese BERT-Networks, EMNLP-IJCNLP.